Software engineering with Semester - 16-week syllabus

Status and prerequisites

Proposed course, subject to instructor and institutional approval. Basic programming, Git and introductory web concepts required. Suggested workload: two instructor-led sessions and 4-6 independent hours weekly; adjust for learner needs. Python 3 runs the small concept drills without third-party packages. The application capstone follows Semester's documented TypeScript direction; learners may use an instructor-approved equivalent typed stack. The Python drills do not replace full-stack labs.

Outcomes

Design contracts; distinguish authentication from authorization; isolate tenants; build accessible workflows; preserve audit and provenance; implement retry-safe operations; operate event delivery; produce a typed SDK; threat-model integrations; reconcile pilots; defend engineering decisions.

Capstone

Build a synthetic Semester planning extension with tenant-scoped tasks, shared-shell UI, typed API client, duplicate-safe events, redacted diagnostics, revocation and certification evidence. No real student data, production credentials or official-record mutations.

Weekly rhythm

Before class: read architecture guide and lab. First session: concepts and guided drill. Second session: capstone implementation and review. End of week: code, visible test evidence, three added boundary tests, design note and 150-250 word reflection. Instructor sets actual dates, late/resubmission rules and office hours. Each starter intentionally raises NotImplementedError.

Assessment

Weekly labs 40%; midterm architecture review 15%; capstone 35%; peer review/reflection 10%. Weekly rubric: correctness 30, tests 25, security/privacy 20, maintainability 15, documentation/reflection 10. Automated grading computes up to 30 correctness points from visible cases; remaining 70 require instructor review. No automatic final grade.

Capstone rubric: functionality 20, authorization/isolation 20, reliability/reconciliation 15, contracts/SDK 15, accessibility 10, evidence/docs 10, defense 10. Midterm rubric: boundary clarity 30, contract quality 25, threat model 25, evidence/communication 20. Peer review: actionable findings 40, justified suggestions 30, respectful collaboration 20, timeliness 10.

Performance levels: exemplary 90-100 (reproducible and justified), proficient 75-89 (complete with minor gaps), developing 60-74 (partial or weak evidence), insufficient below 60 (core requirements missing). These bands are proposed and must match institutional grading policy. Security defects block deployment regardless of academic grade. Remediation is recorded and instructor-approved.

Academic integrity and AI

Cite reused code and permitted assistance; explain understanding in reflection/defense. Never send private records or credentials to AI. Instructor defines permitted assistance for each assessment. No plagiarism allegation or final grade is issued

automatically. Provide accessible alternatives and an appeal route through instructor/institution policy.

Tools and support

Local Python concept runner; Git; instructor-selected TypeScript toolchain and synthetic backing store for capstone. No external service account required for concept drills. Portal is a local HTML prototype with browser-local progress; it has no authentication, shared roster or official grading backend.

Safety

grader.py executes trusted local code only and is not a sandbox. Never run untrusted submissions on a personal or institutional host. Production runners need isolation specified in architecture-guides.md.

Weekly overview

Week 1: System boundaries - Map the shared shell, domain services and private/official data.

Week 2: Git and engineering evidence - Produce a reviewable change with reproducible evidence.

Week 3: Contracts and validation - Validate network-shaped input without coercion.

Week 4: Pagination and API design - Design bounded collection responses.

Week 5: Authentication versus authorization - Require active grants and operation scopes.

Week 6: Tenant isolation - Deny cross-tenant access even with a valid scope.

Week 7: Transactions and audit - Validate mutation evidence before commit.

Week 8: Accessible interfaces - Model accessible errors and explicit interface states.

Week 9: Idempotency - Distinguish first execution, safe replay and conflicts.

Week 10: Events and deduplication - Suppress duplicates while preserving first-seen order.

Week 11: Diagnostics and recovery - Remove sensitive fields from telemetry.

Week 12: Typed SDK behavior - Model bounded retry decisions.

Week 13: Security threat modeling - Reject unsafe webhook destinations in a constrained exercise.

Week 14: Release and certification evidence - Check release-linked evidence completeness.

Week 15: Pilot reconciliation - Identify missing and unexpected canonical identifiers.

Week 16: Capstone activation and defense - Separate evidence from human production authority.

Week 1: System boundaries

Outcome: Map the shared shell, domain services and private/official data.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Return private for draft, official for grade, and synthetic for fixture.

Starter code

```
def classify_record(record):
    """Return private for draft, official for grade, and synthetic for fixture."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"kind": "draft"} | Expected: "private"

Input: {"kind": "grade"} | Expected: "official"

Input: {"kind": "fixture"} | Expected: "synthetic"

Capstone implementation

Draw a context diagram with learner, instructor, publisher and institution boundaries. Add an ADR separating private drafts from official submissions.

Submission checklist

Diagram, ADR-001 and synthetic fixture inventory.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 1 --submission labs/week_01/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 2: Git and engineering evidence

Outcome: Produce a reviewable change with reproducible evidence.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Return True only when title, test_evidence and reviewer are nonempty strings.

Starter code

```
def validate_pr(record):
    """Return True only when title, test_evidence and reviewer are nonempty strings."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"title": "Fix", "test_evidence": "run-1", "reviewer": "A"} | Expected: true

Input: {"title": "Fix", "test_evidence": "", "reviewer": "A"} | Expected: false

Capstone implementation

Implement a small capstone change on a feature branch. Request peer review, respond to comments and record reproducible test evidence.

Submission checklist

PR export, review notes and reproducibility checklist.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 2 --submission labs/week_02/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 3: Contracts and validation

Outcome: Validate network-shaped input without coercion.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Return True when id/title are nonempty strings and status is open or completed.

Starter code

```
def validate_task(record):
    """Return True when id/title are nonempty strings and status is open or completed."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"id": "a", "title": "Task", "status": "open"} | Expected: true

Input: {"id": 1, "title": "Task", "status": "open"} | Expected: false

Input: {"id": "a", "title": "", "status": "bad"} | Expected: false

Capstone implementation

Define Task, Grant, Event and Error contracts. Reject malformed values at the boundary and document nullable versus optional fields.

Submission checklist

Schemas, six invalid-input tests and field dictionary.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 3 --submission labs/week_03/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 4: Pagination and API design

Outcome: Design bounded collection responses.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Given items, offset and limit, return items[offset:offset+limit]; reject negative offset or limit outside 1..100 with ValueError.

Starter code

```
def paginate(record):
    """Given items, offset and limit, return items[offset:offset+limit]; reject negative offset
 or limit outside 1..100 with ValueError."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"items": [1, 2, 3], "offset": 1, "limit": 1} | Expected: [2]

Input: {"items": [], "offset": 0, "limit": 10} | Expected: []

Input: {"items": [1], "offset": 0, "limit": 0} | Expected: "!ValueError"

Capstone implementation

Implement list/create task operations against synthetic storage with cursor pagination, structured errors and bounded inputs.

Submission checklist

Contract examples, handler tests and API walkthrough.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 4 --submission labs/week_04/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 5: Authentication versus authorization

Outcome: Require active grants and operation scopes.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Return True only for active grants containing tasks:read.

Starter code

```
def authorize(record):
    """Return True only for active grants containing tasks:read."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"active": true, "scopes": ["tasks:read"]} | Expected: true

Input: {"active": false, "scopes": ["tasks:read"]} | Expected: false

Input: {"active": true, "scopes": []} | Expected: false

Capstone implementation

Add mock sign-in and scoped request contexts. Keep authentication separate from operation authorization; deny missing or expired grants.

Submission checklist

Identity-flow diagram and denial matrix.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 5 --submission labs/week_05/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 6: Tenant isolation

Outcome: Deny cross-tenant access even with a valid scope.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Require matching nonempty principal/resource tenant, active grant and tasks:read.

Starter code

```
def tenant_access(record):
    """Require matching nonempty principal/resource tenant, active grant and tasks:read."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"principal_tenant": "a", "resource_tenant": "a", "active": true, "scopes": ["tasks:read"]} | Expected: true

Input: {"principal_tenant": "a", "resource_tenant": "b", "active": true, "scopes": ["tasks:read"]} | Expected: false

Input: {"principal_tenant": "", "resource_tenant": "", "active": true, "scopes": ["tasks:read"]} | Expected: false

Capstone implementation

Run the same identifiers in two tenants. Test database, API, cache and search boundaries; for a mock layer clearly state what is not tested.

Submission checklist

Isolation matrix with actual results and limitations.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 6 --submission labs/week_06/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 7: Transactions and audit

Outcome: Validate mutation evidence before commit.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Return True only if actor, tenant, action, target and request_id are nonempty strings.

Starter code

```
def audit_ready(record):
    """Return True only if actor, tenant, action, target and request_id are nonempty
strings."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"actor": "u", "tenant": "t", "action": "create", "target": "x", "request_id": "r"} | Expected: true

Input: {"actor": "u", "tenant": "t", "action": "create", "target": "x"} | Expected: false

Capstone implementation

Add migration constraints, transactional audit metadata and a rollback plan. Demonstrate that failed audit writes prevent protected mutations.

Submission checklist

Migration, rollback procedure and failure test.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 7 --submission labs/week_07/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 8: Accessible interfaces

Outcome: Model accessible errors and explicit interface states.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Return {field, message, described_by}; described_by is field + -error.

Starter code

```
def error_model(record):
    """Return {field, message, described_by}; described_by is field + -error."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"field": "email", "message": "Required"} | Expected: {"field": "email", "message": "Required", "described_by": "email-error"}

Capstone implementation

Build keyboard-operable task forms with labels, error associations, focus handling and explicit loading/empty/error/offline states.

Submission checklist

UI demo, manual accessibility checklist and state matrix.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 8 --submission labs/week_08/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 9: Idempotency

Outcome: Distinguish first execution, safe replay and conflicts.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Given previous_digest (null or string) and digest return execute, replay or conflict.

Starter code

```
def idempotency(record):
    """Given previous_digest (null or string) and digest return execute, replay or conflict."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"previous_digest": null, "digest": "a"} | Expected: "execute"

Input: {"previous_digest": "a", "digest": "a"} | Expected: "replay"

Input: {"previous_digest": "a", "digest": "b"} | Expected: "conflict"

Capstone implementation

Implement idempotency key reservation and request digest conflict handling. Simulate two concurrent identical requests.

Submission checklist

Concurrency test, replay policy and retention decision.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 9 --submission labs/week_09/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 10: Events and deduplication

Outcome: Suppress duplicates while preserving first-seen order.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Given ids list return distinct ids in first-seen order.

Starter code

```
def deduplicate(record):
    """Given ids list return distinct ids in first-seen order."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"ids": ["a", "b", "a", "c"]} | Expected: ["a", "b", "c"]

Input: {"ids": []} | Expected: []

Capstone implementation

Add an outbox and event consumer. Test duplicate and stale events, bounded retries and a dead-letter recovery path.

Submission checklist

Event contract, consumer tests and reconciliation instructions.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 10 --submission labs/week_10/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 11: Diagnostics and recovery

Outcome: Remove sensitive fields from telemetry.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Return record excluding token, password, student_name and submission_body keys.

Starter code

```
def redact(record):
    """Return record excluding token, password, student_name and submission_body keys."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"token": "secret", "request_id": "r", "student_name": "X"} | Expected: {"request_id": "r"}

Input: {"status": 503} | Expected: {"status": 503}

Capstone implementation

Add redacted request tracing and dependency outage simulation. Recover without exposing private payloads or silently dropping writes.

Submission checklist

Failure timeline, telemetry sample and recovery evidence.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 11 --submission labs/week_11/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 12: Typed SDK behavior

Outcome: Model bounded retry decisions.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Return True for 429 or 503 only when attempt < max_attempts and method GET or idempotency_key is nonempty.

Starter code

```
def retry_allowed(record):
    """Return True for 429 or 503 only when attempt < max_attempts and method GET or
idempotency_key is nonempty."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"status": 503, "attempt": 0, "max_attempts": 3, "method": "GET"} | Expected: true

Input: {"status": 503, "attempt": 0, "max_attempts": 3, "method": "POST"} | Expected: false

Input: {"status": 403, "attempt": 0, "max_attempts": 3, "method": "GET"} | Expected: false

Capstone implementation

Implement a strict TypeScript client or equivalent typed wrapper with runtime validation, cancellation and bounded safe retries.

Submission checklist

Client code, type-misuse tests and compatibility table.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 12 --submission labs/week_12/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 13: Security threat modeling

Outcome: Reject unsafe webhook destinations in a constrained exercise.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

For literal-IP HTTPS URLs only, return True if IP is globally routable, no credentials and port absent or 443. DNS hostnames are intentionally rejected.

Starter code

```
def safe_destination(record):
    """For literal-IP HTTPS URLs only, return True if IP is globally routable, no credentials
and port absent or 443. DNS hostnames are intentionally rejected."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"url": "https://8.8.8.8/hook"} | Expected: true

Input: {"url": "https://127.0.0.1/hook"} | Expected: false

Input: {"url": "http://8.8.8.8/hook"} | Expected: false

Input: {"url": "https://example.com/hook"} | Expected: false

Capstone implementation

Threat-model object authorization and webhook egress. Explain DNS rebinding, redirects and the limits of the literal-IP teaching exercise.

Submission checklist

Threat model, attack tests and unresolved risk register.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 13 --submission labs/week_13/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 14: Release and certification evidence

Outcome: Check release-linked evidence completeness.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Require artifact_digest, contract_version, fixture_version plus passing contract, isolation, revocation, accessibility and recovery evidence.

Starter code

```
def evidence_ready(record):
    """Require artifact_digest, contract_version, fixture_version plus passing contract,
isolation, revocation, accessibility and recovery evidence."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"artifact_digest": "d", "contract_version": "1", "fixture_version": "1", "tests": {"contract": true, "isolation": true, "revocation": true, "accessibility": true, "recovery": true}} | Expected: true

Input: {"artifact_digest": "d", "tests": {}} | Expected: false

Capstone implementation

Freeze a release artifact and assemble contract, isolation, revocation, accessibility and recovery evidence tied to its digest.

Submission checklist

Evidence manifest, dependency inventory and mock certification submission.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 14 --submission labs/week_14/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 15: Pilot reconciliation

Outcome: Identify missing and unexpected canonical identifiers.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Given expected and observed ids return sorted missing and unexpected lists.

Starter code

```
def reconcile(record):
    """Given expected and observed ids return sorted missing and unexpected lists."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"expected": ["a", "b"], "observed": ["b", "c"]} | Expected: {"missing": ["a"], "unexpected": ["c"]}

Input: {"expected": ["a"], "observed": ["a"]} | Expected: {"missing": [], "unexpected": []}

Capstone implementation

Run a synthetic pilot, inject drift, reconcile identifiers and rehearse rollback. Record support ownership and unresolved defects.

Submission checklist

Pilot report, reconciliation output and rollback rehearsal.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 15 --submission labs/week_15/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Week 16: Capstone activation and defense

Outcome: Separate evidence from human production authority.

Estimated work: 4-6 independent hours plus instructor sessions. Suggested allocation: concept drill 1 hour; capstone 2-3 hours; testing/review/reflection 1-2 hours.

Required concept drill

Require certified, institution_approved, reconciled, rollback_ready and support_ready all exactly True.

Starter code

```
def activation_ready(record):
    """Require certified, institution_approved, reconciled, rollback_ready and support_ready
all exactly True."""
    raise NotImplementedError("Implement this lab")
```

Visible grading cases

Input: {"certified": true, "institution_approved": true, "reconciled": true, "rollback_ready": true, "support_ready": true} | Expected: true

Input: {"certified": true, "institution_approved": false, "reconciled": true, "rollback_ready": true, "support_ready": true} | Expected: false

Capstone implementation

Demonstrate the full extension, revoke access and show recovery/offboarding. Defend three architecture decisions and publish only chosen synthetic portfolio evidence.

Submission checklist

Capstone demo, defense notes, portfolio consent and final evidence pack.

Submit code, three additional boundary/denial tests, test evidence, DESIGN.md and a 150-250 word reflection. Clearly label mocked layers and untested assumptions.

Assessment

Correctness 30; tests 25; security/privacy 20; maintainability 15; documentation/reflection 10. Automated visible cases generate correctness feedback only. Instructor reviews the capstone increment and remaining dimensions. No automated final grade.

Local verification

python grader.py --lab 16 --submission labs/week_16/starter.py --report result.json

Safety and limitations

Starter intentionally fails until implemented. Run only trusted local code. Grader is not a security sandbox. No student records or production secrets. Python drill models a concept; full application work is required separately.

Review prompts

Which trust boundary is crossed? Which failure or invalid-input cases matter? What evidence distinguishes the implemented layer from a mock? How would this feature behave after grant revocation?

Partner certification workflow - proposed operating procedure

Certification is release-specific and does not grant institutional authority. No SLA or legal timeline is adopted by this document.

1. Register publisher

Owner: Partner operations

Action: Create publisher identity, educational purpose and ownership record.

Evidence: Legal entity or accountable institutional unit; organization administrator.

Exit gate: Publisher record exists without institutional data access.

Failure handling: Unclear ownership: remain draft.

2. Verify ownership and contacts

Owner: Partnerships

Action: Verify organization control and technical, support and incident contacts.

Evidence: Verification record, contact acknowledgment and escalation route.

Exit gate: Named accountable contacts confirmed.

Failure handling: Failed verification: reject or request clarification.

3. Declare use case and domain boundaries

Owner: Partner product lead

Action: Identify users, benefits, domain authority, prohibited uses and private/official record boundaries.

Evidence: Use-case diagram and intended-purpose statement.

Exit gate: No unapproved authority transfer.

Failure handling: Consequential decision claims require redesign.

4. Register application and environments

Owner: Partner engineer

Action: Create immutable app identity and separate sandbox/staging configuration.

Evidence: Application record and environment inventory.

Exit gate: Credentials and fixtures isolated by environment.

Failure handling: Cross-environment access blocks progression.

5. Inventory data and subprocessors

Owner: Privacy reviewer

Action: Record collection, destinations, retention, deletion, hosting and subprocessors.

Evidence: Data-flow diagram and reviewed retention/deletion schedule.

Exit gate: Purpose-limited minimization established.

Failure handling: Unresolved legal placeholders remain blockers.

6. Request least-privilege permissions

Owner: Partner engineer and reviewer

Action: Map each operation and data category to a purpose; remove unnecessary requests.

Evidence: Permission-purpose matrix and user-consent requirements.

Exit gate: All permissions justified; no blanket tenant access.

Failure handling: Excess permissions return for revision.

7. Provision synthetic sandbox

Owner: Platform engineer

Action: Provision deterministic tenant fixtures and versioned reset capability.

Evidence: Fixture version, setup log and reset test.

Exit gate: No production records; two-tenant test available.

Failure handling: Provisioning failure creates owned support task.

8. Run contract conformance

Owner: Partner engineer

Action: Test request/response schemas, errors, pagination, idempotency and event compatibility.

Evidence: Release-linked test results and executable examples.

Exit gate: Required cases pass for the submitted contract.

Failure handling: Failure creates specific remediation findings.

9. Prove tenant and resource isolation

Owner: Security reviewer

Action: Test cross-user/app/tenant/environment access and property restrictions.

Evidence: Negative tests across API, storage, database, cache and search.

Exit gate: No unauthorized disclosure or mutation.

Failure handling: Any leak blocks certification.

10. Review identity, credentials and egress

Owner: Security reviewer

Action: Review token validation, rotation, revocation, secret handling and destination protections.

Evidence: Threat model and credential/SSRF test evidence.

Exit gate: Critical risks resolved; residual risk documented.

Failure handling: No self-approval; security exceptions need named authority.

11. Review accessible user workflows

Owner: Accessibility reviewer

Action: Test keyboard, focus, labels, errors and assistive-technology workflows.

Evidence: Automated results plus manual review and remediation.

Exit gate: Required workflows meet approved accessibility criteria.

Failure handling: Untested workflows cannot receive a pass badge.

12. Exercise operations and offboarding

Owner: Operations reviewer

Action: Inject outage and duplicate events; rehearse support, export, revocation, downstream deletion and reconciliation.

Evidence: Recovery report and offboarding acknowledgment evidence.

Exit gate: Bounded recovery works; owners and limitations explicit.

Failure handling: Unknown downstream deletion prevents completion claims.

13. Freeze release and submit evidence

Owner: Partner release owner

Action: Bind artifact digest, contract, dependencies, hosting, permissions and evidence.

Evidence: Immutable manifest and declared limitations.

Exit gate: Evidence matches exact release candidate.

Failure handling: Material changes invalidate relevant evidence.

14. Make independent certification decision

Owner: Authorized review panel

Action: Review findings and issue approved, conditional, changes-required or rejected decision.

Evidence: Reviewer identities, reasoning, conditions and expiration.

Exit gate: Decision authorized and auditable; no self-certification.

Failure handling: Open critical findings block approval.

15. Approve institution-specific pilot

Owner: Institutional approver

Action: Approve installation, minimum grants, users, pilot boundaries, success criteria and rollback.

Evidence: Institution decision and pilot plan; user consent where needed.

Exit gate: Publisher certification does not substitute for institution approval.

Failure handling: Absent authority leaves installation unenabled.

16. Reconcile pilot and activate or reject

Owner: Institution and platform operations

Action: Compare expected/observed outcomes, resolve drift, verify rollback/support and decide bounded activation.

Evidence: Reconciliation report, activation decision and support readiness.

Exit gate: Production-enabled status only after verified activation.

Failure handling: Failed pilot returns to remediation or offboarding.

Renewal and change control

Review again when scopes, hosting, subprocessors, material dependencies, data practices or reviewed workflows change. Record certification status separately from application, release and tenant-installation states. Revocation immediately blocks access; downstream deletion acknowledgment is separately tracked. Security incidents may trigger suspension with preserved evidence and a documented restoration decision.

Semester architecture guides - proposed implementation baseline

Evidence and status

The uploaded docs/platform/DEVELOPER_PLATFORM_AND_MARKETPLACE.md describes a design-stage platform and no published API. docs/guides/Developer Guide.html documents strict TypeScript, Zod validation, user-scoped access, worker-only service roles and audited mutations. This pack does not establish current live repository status. No certifications or production guarantees are claimed.

1. System and trust boundaries

Reuse the Semester account, Action Center, search, workspace and design system. The developer console is a control plane, not unrestricted database administration. Public documentation, publisher organization membership, app permissions and institutional approval are separate boundaries. Learning drafts remain private until an authorized submission; official grades require institutional authority. Native domain services own canonical entities; connector adapters carry source identifiers, freshness and reconciliation evidence.

2. Gateway request sequence

TLS/edge limits -> route/environment resolution -> content and size validation -> token verification -> principal/app resolution -> active installation/grant -> tenant/purpose policy -> service object/property policy -> tenant-aware transaction -> audit/outbox -> response allowlist -> redacted telemetry. Database RLS is defense in depth. Never authorize from a supplied tenant header alone. Fail closed if required policy dependencies are unavailable. Protected mutation audit metadata commits transactionally.

3. Identity profiles

Portal: server-managed session/BFF, secure cookies and CSRF defenses. Public native clients: authorization code with PKCE S256. Confidential clients: code/PKCE plus supported client authentication. Machine integrations: approved client-credentials profile, narrowly scoped to environment and installation. Workers: dedicated workload identity. No implicit or password grants. Validate signatures, allowed algorithms, issuer, audience, token type and time claims; ID tokens are not access tokens. Key discovery uses trusted issuer configuration, not token-supplied URLs. Publish token lifetimes and refresh/rotation rules only after review.

4. Authorization and revocation

Evaluate identity, app, environment, installation, active grant, tenant, purpose, resource and property policy. Scope alone is insufficient. Tenant/environment appear in cache keys, storage policy and cursor bindings. Reauthorize queued jobs, retries and replay. Revocation invalidates caches, blocks issuance/access, pauses subscriptions, stops queued work and starts offboarding. Measure propagation before adopting a target.

5. API contract rules

The accompanying OpenAPI 3.1 contract proposes 67 operations. Base URLs are deliberately absent; identity.example.invalid is a placeholder. Errors carry code/message/retryable/requestId. Collections have opaque cursors bound to query and authorization context. Writes require idempotency keys bound to principal/app/tenant/environment/operation/request digest. Same key/different digest conflicts. Store result atomically with the operation. PATCH uses If-Match; successful mutable reads and writes must publish ETag in implementation. Schema fields not in the allowlist are rejected. Membership and credential

changes require recent reauthentication. No public generic state setter may bypass approval workflows.

6. Events and webhooks

Use a transactional outbox and bounded workers. Assume duplicates and no global ordering. Envelope: eventId, type, schemaVersion, resourceId, resourceVersion, occurredAt and provenance; include minimal payload. Sign raw bytes with a reviewed signature scheme, timestamp and key identifier; recipients enforce replay windows and deduplication. Publish canonical signing rules before implementation. Constrained egress validates destination at registration and connect time, rejects internal/private/link-local/metadata destinations, handles DNS rebinding, disables redirects and bounds responses. The week-13 literal-IP drill is not a production SSRF defense.

7. SDK design

Proposed packages: @semester/contracts, @semester/sdk, @semester/sdk-server, @semester/testing. Generate transport types from an approved contract; validate unknown network values at runtime. strict, noUncheckedIndexedAccess, exactOptionalPropertyTypes and useUnknownInCatchVariables enabled. Branded identifiers distinguish resources but do not authorize. Browser package excludes secrets and server dependencies. Inject token providers; no automatic persistent storage. Retry only transient operations with explicit idempotency for writes, bounded attempts/elapsed time and Retry-After handling. Support cancellation and timeouts. Contract drift, type misuse and browser-bundle leakage tests gate release.

8. Learning and grading boundary

Learner code never executes in the portal server. Production grading requires disposable isolated workers, no credentials, network disabled by default, bounded CPU/memory/time/output, no host mounts or privileged runtime, pinned dependencies, inaccessible hidden tests and cleanup. The included grader is ONLY for trusted local exercises: a subprocess timeout is not a sandbox. Learner code can access the local account and tamper with tests; therefore results are instructional feedback, not trusted institutional assessment.

9. AI policy

Existing project standard says no student records to AI. Use approved documentation and synthetic fixtures only. AI cannot grant permissions, certify, publish official grades or independently change registration. Student declarations explain allowed assistance; instructor remains assessment authority.

10. Release and recovery

Inventory current repository before introducing duplicate infrastructure. CI gates types, contracts, unit tests, RLS, negative authorization, end-to-end and accessibility. Preview/staging are not production. Evidence ties digest, environment, test version and verifier. Activation requires certification, institution decision, reconciled pilot, rollback and support. Recovery drills include connector outage, duplicate events, partial import and grant revocation. Do not promise recovery metrics until measured.

11. Contract omissions and decisions before production

This control-plane contract does not implement OAuth endpoints, binary artifact uploads, marketplace billing, domain-record APIs or curriculum APIs. Evidence references artifacts provisioned by a separately reviewed artifact service. Signing algorithm, lifetimes, retention windows, rate quotas, incident timelines and legal terms remain approval decisions. Full OpenAPI semantic validation and implementation conformance are still required.

Sources

Project: docs/platform/DEVELOPER_PLATFORM_AND_MARKETPLACE.md; docs/guides/Developer Guide.html.

OAuth security baseline: RFC 9700 (https://www.rfc-editor.org/rfc/rfc9700.html).

API risk baseline: OWASP API Security Top 10 2023 (https://owasp.org/API-Security/editions/2023/en/0x11-t10/).

These are references for implementation review, not claims of compliance.

Automated grading script

This script is also supplied in the build pack. Trusted local exercises only. Do not use for adversarial submissions or official grading without isolated infrastructure and independent evidence verification.

```
import argparse, json, subprocess, sys, tempfile
from pathlib import Path
p=argparse.ArgumentParser(description="LOCAL TRUSTED CODE ONLY. This is not a security sandbox.")
p.add_argument('--lab',type=int,required=True,choices=range(1,17))
p.add_argument('--submission',required=True)
p.add_argument('--report',required=True)
a=p.parse_args()
root=Path(__file__).resolve().parent
spec=json.loads((root/'labs'/f'week_{a.lab:02d}'/'tests.json').read_text())
source=Path(a.submission).read_text()
runner="""import json,sys
ns={}
exec(compile(sys.stdin.readline(),'<submission>','exec'),ns)
spec=json.loads(sys.stdin.read())
results=[]
for case in spec['cases']:
    try:
        actual=ns[spec['function']](case['input'])
        passed=actual==case['expected']
    except Exception as e:
        passed=case['expected']=='!ValueError' and type(e) is ValueError
    results.append(bool(passed))
print(json.dumps(results))
"""
# JSON-encoded source avoids multiline transport ambiguity.
runner=runner.replace("exec(compile(sys.stdin.readline(),",
"exec(compile(json.loads(sys.stdin.readline()),")
try:
    run=subprocess.run([sys.executable,'-I','-
c',runner],input=json.dumps(source)+'\n'+json.dumps(spec),text=True,capture_output=True,timeout=10)
    results=json.loads(run.stdout) if run.returncode==0 else []
    passed=sum(x is True for x in results)
    error=None if run.returncode==0 else 'Execution failed'
except (subprocess.TimeoutExpired,json.JSONDecodeError):
    passed=0; results=[]; error='Timeout or invalid output'
total=len(spec['cases'])
report={'lab':a.lab,'passed':passed,'total':total,'correctness_points':round(30*passed/total,2),'man
ual_points_available':70,'final_grade':None,'error':error,'warning':'Trusted local code only; not
isolated from host files/network. Production requires an isolated runner.'}
Path(a.report).write_text(json.dumps(report,indent=2))
print(json.dumps(report))
sys.exit(0 if passed==total else 1)
```

Developer console API index

The accompanying machine-readable OpenAPI JSON provides request/response schemas, scope requirements, pagination, idempotency, authority notes and errors. This PDF index is not a replacement for the contract.

GET /v1/organizations | organizations:read

POST /v1/organizations | organizations:write

GET /v1/organizations/{organizationId} | organizations:read

GET /v1/organizations/{organizationId}/members | memberships:read

POST /v1/organizations/{organizationId}/members | memberships:write

PATCH /v1/organizations/{organizationId}/members/{membershipId} | memberships:write

POST /v1/organizations/{organizationId}/members/{membershipId}/revocations | memberships:write

GET /v1/organizations/{organizationId}/apps | apps:read

POST /v1/organizations/{organizationId}/apps | apps:write

GET /v1/apps/{appId} | apps:read

PATCH /v1/apps/{appId} | apps:write

POST /v1/apps/{appId}/retirements | apps:write

GET /v1/apps/{appId}/environments | environments:read

POST /v1/apps/{appId}/environments | environments:write

GET /v1/apps/{appId}/environments/{environmentId} | environments:read

POST /v1/apps/{appId}/environments/{environmentId}/resets | environments:write

GET /v1/apps/{appId}/environments/{environmentId}/credentials | credentials:read

POST /v1/apps/{appId}/environments/{environmentId}/credentials | credentials:write

POST /v1/apps/{appId}/environments/{environmentId}/credentials/{credentialId}/rotations | credentials:write

POST /v1/apps/{appId}/environments/{environmentId}/credentials/{credentialId}/revocations | credentials:write

GET /v1/permissions | permissions:read

POST /v1/installations/{installationId}/grant-requests | grants:request

GET /v1/grant-requests/{grantRequestId} | grants:read

POST /v1/grant-requests/{grantRequestId}/decisions | grants:approve

GET /v1/installations/{installationId}/grants | grants:read

GET /v1/grants/{grantId} | grants:read

POST /v1/grants/{grantId}/revocations | grants:revoke

GET /v1/event-types | events:read

GET /v1/apps/{appId}/environments/{environmentId}/webhook-subscriptions | webhooks:read

POST /v1/apps/{appId}/environments/{environmentId}/webhook-subscriptions | webhooks:write

GET /v1/webhook-subscriptions/{subscriptionId} | webhooks:read

PATCH /v1/webhook-subscriptions/{subscriptionId} | webhooks:write

POST /v1/webhook-subscriptions/{subscriptionId}/verifications | webhooks:write

POST /v1/webhook-subscriptions/{subscriptionId}/revocations | webhooks:write

GET /v1/webhook-subscriptions/{subscriptionId}/deliveries | webhooks:read

POST /v1/webhook-subscriptions/{subscriptionId}/replays | webhooks:write

GET /v1/apps/{appId}/releases | releases:read

POST /v1/apps/{appId}/releases | releases:write

GET /v1/releases/{releaseId} | releases:read

POST /v1/releases/{releaseId}/withdrawals | releases:write

GET /v1/releases/{releaseId}/evidence | evidence:read

POST /v1/releases/{releaseId}/evidence | evidence:write

POST /v1/releases/{releaseId}/test-runs | tests:write

GET /v1/test-runs/{testRunId} | tests:read

GET /v1/apps/{appId}/certification-submissions | certification:read

POST /v1/apps/{appId}/certification-submissions | certification:write

GET /v1/certification-submissions/{certificationId} | certification:read

GET /v1/certification-submissions/{certificationId}/findings | certification:read

POST /v1/certification-submissions/{certificationId}/findings | certification:review

POST /v1/certification-submissions/{certificationId}/decisions | certification:decide

GET /v1/apps/{appId}/installations | installations:read

POST /v1/apps/{appId}/installations | installations:write

GET /v1/installations/{installationId} | installations:read

POST /v1/installations/{installationId}/approvals | installations:approve

POST /v1/installations/{installationId}/pilots | pilots:write

GET /v1/pilots/{pilotId} | pilots:read

POST /v1/installations/{installationId}/activations | installations:activate

POST /v1/installations/{installationId}/offboarding | offboarding:write

GET /v1/offboarding/{offboardingId} | offboarding:read

POST /v1/apps/{appId}/exports | exports:write

GET /v1/jobs/{jobId} | jobs:read

GET /v1/organizations/{organizationId}/audit-events | audit:read

GET /v1/apps/{appId}/environments/{environmentId}/usage | usage:read

GET /v1/apps/{appId}/support-cases | support:read

POST /v1/apps/{appId}/support-cases | support:write

GET /v1/support-cases/{supportCaseId} | support:read

GET /v1/capabilities | capabilities:read

Verification record

16 instructor reference exercises passed 40 visible cases through the grading script. Starter code intentionally contains unimplemented functions. OpenAPI operation identifiers and local schema references checked; full semantic validation not performed. Browser interaction and production deployment not tested.